EX288

Guide complet unifié

Labs chronométrés, corrigés types, checklists, dépannage rapide et examens blancs

Examen ciblé	Red Hat Certified OpenShift Application Developer - EX288
Positionnement	Pack d'entraînement pratique, fusionnant le socle V4 et les enrichissements V5 dans un document unique.
Usage conseillé	Préparation intensive, auto-correction, répétition chronométrée et simulation d'épreuve.

Principe directeur - Ce document garde l'ADN de la V4 - priorité aux gestes observables de l'examen - tout en ajoutant ce qui manquait pour un support quasi autosuffisant : solutions types minimales, checklists de validation, méthodes de diagnostic, scénarios blancs et plans d'entraînement.

Références de base utilisées pour ce guide : page officielle EX288 et offre DO289/DO288, dans la continuité du guide source.

Sommaire

- 1. Positionnement et mode d'emploi
- 2. Matrice de couverture et méthode d'examen
- 3. Les 12 labs chronométrés
- 4. Dépannage rapide et séquence de diagnostic
- 5. Examens blancs A et B
- 6. Plans d'entraînement
- 7. Commandes réflexes

1. Positionnement et mode d'emploi

Ce guide est conçu comme un support d'entraînement opérationnel. Il privilégie les gestes attendus à l'examen : créer, corriger, valider, relancer puis revalider. La logique reste volontairement orientée terrain : résultat observable, contrôle rapide et réduction des erreurs répétitives.

Ce que ce document est

- un pack de préparation intensive centré sur l'exécution et la vérification ;
- un guide unique qui regroupe le socle V4 et les enrichissements V5 ;
- un support de révision rapide avant inscription ou avant simulation blanche ;

Ce que ce document n'est pas

- un cours théorique exhaustif sur OpenShift ;
- une transcription de la documentation officielle ;
- un substitut à la pratique réelle au clavier.

Comment utiliser ce guide

1. Mode découverte : lire un lab, comprendre son objectif, reproduire sans contrainte de temps.
2. Mode entraînement : exécuter le lab dans le temps cible, puis cocher la checklist.
3. Mode examen blanc : enchaîner plusieurs thèmes sans aide, puis analyser les erreurs et la commande qui a débloqué la situation.

Règle d'or - Règle d'or : après chaque action importante, vérifie immédiatement l'effet obtenu avec oc get, oc describe, les logs, les events et l'état de la route. Une modification non testée devient une dette de temps.

2. Matrice de couverture et méthode d'examen

La matrice ci-dessous conserve l'esprit de la V4, mais ajoute les signaux de validation, l'artefact principal à maîtriser et le piège dominant par famille de sujet.

Thème	Labs	Niveau	Signal de validation	Artefact	Piège dominant

Projets, applications, probes, route	Lab 1, 2, 3	Critique	Pods Ready, route OK, probes vertes	Deployment / Service / Route	Namespace ou port incohérent
Helm et Kustomize	Lab 9	Élevé	Rendu final conforme, overlay appliqué	Chart + patch	Conflit values / overlay
Images, builds, registry interne	Lab 4, 5	Critique	Image disponible puis déployée	ImageStream / BuildConfig	Ancienne image toujours utilisée
S2I et builder custom	Lab 6	Critique	Build OK puis runtime sain	BuildConfig S2I	Confusion build OK / app KO
Triggers, hooks, relances	Lab 7	Élevé	Déclenchement réel observé	Trigger / hook	Événement non déclenché
Templates OpenShift	Lab 8	Élevé	Paramètres réellement consommés	Template	Valeur codée en dur
Pipelines Tekton	Lab 10	Critique	PipelineRun OK, logs lisibles	PipelineRun / TaskRun	Workspace ou SA manquant
Operators	Lab 11	Moyen à critique	CR acceptée et ressources créées	CRD / CR	apiVersion, kind ou namespace faux
Dépannage transversal	Lab 12 + section pannes	Critique	Cause racine identifiée et corrigée	Méthode de diagnostic	Traitement du symptôme avant la cause

Méthode d'examen

4. Lire tout le sujet pendant 10 à 15 minutes pour repérer les tâches rapides et les points risqués.
5. Sécuriser d'abord les points les plus accessibles : projet, déploiement, service, route, configuration simple.
6. Quand un diagnostic s'enlise, passer temporairement au sujet suivant après avoir noté le symptôme et la dernière commande utile.
7. Garder 15 minutes finales pour revalider les routes, les pods, les builds, les triggers et les objets qui doivent être observables.

Déclic examen - Le bon réflexe n'est pas d'écrire plus de YAML. Le bon réflexe est d'obtenir le résultat attendu avec le minimum de correction nécessaire puis de le prouver.

3. Les 12 labs chronométrés

Les douze labs ci-dessous reprennent la progression de la V4 et y ajoutent, pour chaque sujet, une solution type minimale, une checklist de validation et une variante d'entraînement.

Lab 1 - Projet, application simple, Service, Route, probes

Temps cible : 20 min

Objectif : Déployer rapidement une application mono-conteneur proprement exposée.

Point de départ

Projet vide ; image HTTP de démonstration disponible dans le registre.

Points d'attention

- oubli de namespace ;
- port de conteneur incohérent avec le Service ;
- route créée avant un service correct ;
- probes trop agressives.

Tâches à exécuter

8. Créer le projet ex288-lab1.
9. Déployer l'application dans le namespace visé.
10. Créer un Service puis une Route.
11. Ajouter readiness et liveness probes cohérentes.
12. Vérifier l'URL et corriger jusqu'à obtenir une réponse HTTP stable.

Commandes et vérifications réflexes

- oc project ex288-lab1
- oc get all -n ex288-lab1
- oc describe deploy/<nom>
- oc get route
- oc logs deploy/<nom> --tail=50

Solution type minimale

Créer d'abord le projet, puis un Deployment minimal, ensuite le Service exposant le bon targetPort, puis la Route. Ajouter des probes simples sur le bon chemin et avec un délai initial raisonnable. Valider en priorité l'état Ready, la cohérence des ports et la réponse HTTP.

Checklist de validation

- ☐ le bon namespace est actif ;
- ☐ au moins un pod est Running et Ready ;
- ☐ le Service pointe sur le bon port ;
- ☐ la Route répond ;
- ☐ les logs ne montrent pas de restart anormal ;

Variante examen

Même exercice mais avec image différente, port non standard ou chemin de probe spécifique.

Erreurs fréquentes et correction rapide

Route KO : vérifier Service, targetPort et endpoints. Pod Running mais non Ready : comparer le port et le path des probes avec l'application réelle.

Lab 2 - Multi-conteneurs dans un même Pod

Temps cible : 25 min

Objectif : Monter un déploiement avec conteneur principal et sidecar utilitaire.

Point de départ

Image applicative et image sidecar déjà disponibles ; le sidecar sert de collecteur ou de proxy local.

Points d'attention

- sidecar qui crash ;
- mauvaise commande d'entrée ;
- volume mounted sur un seul conteneur ;
- confusion entre port exposé et targetPort.

Tâches à exécuter

13. Créer le déploiement avec deux conteneurs dans le même Pod.
14. Partager un volume emptyDir entre eux.
15. Vérifier que le conteneur secondaire n'empêche pas le Pod de devenir Ready.
16. Exposer le bon port via Service et Route.

Commandes et vérifications réflexes

- oc get pods -o wide
- oc describe pod/<pod>
- oc logs <pod> -c <main>
- oc logs <pod> -c <sidecar>
- oc exec <pod> -c <main> -- ls -l /tmp

Solution type minimale

Définir clairement le conteneur applicatif et le sidecar, leur montage commun sur emptyDir, puis vérifier les logs séparément par conteneur. Le Service doit viser le port du conteneur principal, pas celui du sidecar.

Checklist de validation

- ☐ le pod contient bien deux conteneurs ;
- ☐ le volume partagé est visible des deux côtés ;
- ☐ le sidecar n'entraîne pas de CrashLoopBackOff ;
- ☐ le pod passe à Ready ;
- ☐ le trafic réseau arrive bien sur le conteneur principal ;

Variante examen

Sidecar avec commande cassée à corriger ou montage partagé manquant au départ.

Erreurs fréquentes et correction rapide

Pod non Ready : isoler les logs du sidecar puis du conteneur principal. Service en erreur : confirmer le bon targetPort.

Lab 3 - ConfigMaps, Secrets, variables et montages

Temps cible : 20 min

Objectif : Injecter proprement de la configuration et des secrets dans une application.

Point de départ

L'application attend des variables d'environnement et un fichier de configuration.

Points d'attention

- clé mal orthographiée ;
- secret monté au mauvais chemin ;
- application qui lit seulement au démarrage ;
- rollout oublié après changement.

Tâches à exécuter

17. Créer une ConfigMap à partir de valeurs simples ou d'un fichier.
18. Créer un Secret opaque.
19. Injecter une partie en variables d'environnement et une partie en volume.
20. Redéployer et vérifier que l'application consomme bien les valeurs.

Commandes et vérifications réflexes

- oc get cm,secret
- oc describe deploy/<nom>
- oc exec <pod> -- printenv | sort
- oc exec <pod> -- cat /etc/config/app.properties

Solution type minimale

Séparer clairement ce qui doit être injecté en variables d'environnement de ce qui doit être monté en fichier. Après modification, forcer un rollout si l'application lit la configuration seulement au démarrage.

Checklist de validation

- ☐ la ConfigMap existe ;
- ☐ le Secret existe ;
- ☐ les variables attendues sont visibles dans le pod ;
- ☐ le fichier monté est lisible au bon chemin ;
- ☐ le redéploiement a bien pris en compte la nouvelle config ;

Variante examen

Configuration mixte env + fichier + secret binaire ou nom de clé volontairement faux.

Erreurs fréquentes et correction rapide

L'application ne voit rien : vérifier le nom exact des clés et le mountPath. Valeurs inchangées : relancer un rollout ou vérifier si le pod a bien été recréé.

Lab 4 - ImageStreams et registre interne

Temps cible : 25 min

Objectif : Créer un flux image maîtrisé avec tag et déclenchement propre.

Point de départ

Une image source existe. Il faut la référencer dans OpenShift puis la consommer.

Points d'attention

- confusion autour de latest ;

- image importée mais déploiement toujours pointé sur l'ancienne référence ;
- rollout non déclenché.

Tâches à exécuter

21. Créer un ImageStream personnalisé.
22. Importer ou taguer une image dans l'ImageStream.
23. Déployer l'application depuis cet ImageStream.
24. Changer le tag et confirmer le comportement attendu.

Commandes et vérifications réflexes

- oc get is,istag
- oc describe is/<nom>
- oc set image ...
- oc rollout status deploy/<nom>

Solution type minimale

Structurer le flux source -> ImageStream -> tag -> déploiement. Toujours confirmer que le déploiement consomme l'ImageStreamTag attendu, puis vérifier si un trigger doit provoquer ou non le rollout.

Checklist de validation

- ☐ ImageStream présent ;
- ☐ tag correctement créé ou importé ;
- ☐ déploiement relié à la bonne image ;
- ☐ nouveau tag visible ;
- ☐ comportement du rollout compris et observé ;

Variante examen

Changement de tag sans trigger, ou trigger d'image à activer.

Erreurs fréquentes et correction rapide

Ancienne version toujours servie : contrôler ImageStreamTag, image du Deployment et historique de rollout.

Lab 5 - BuildConfig depuis Containerfile/Dockerfile

Temps cible : 25 min

Objectif : Construire une image avec BuildConfig puis l'utiliser.

Point de départ

Un dépôt Git avec un Containerfile ou Dockerfile minimal est fourni.

Points d'attention

- mauvaise branche Git ;
- fichier de build introuvable ;
- push vers mauvais registre ;
- build OK mais déploiement inchangé.

Tâches à exécuter

25. Créer la ressource BuildConfig adaptée.

26. Lancer le build et suivre les logs.
27. Vérifier la publication dans le registre interne.
28. Déployer l'image construite.

Commandes et vérifications réflexes

- oc start-build <buildconfig> --follow
- oc get builds
- oc logs build/<nom-build>
- oc get is

Solution type minimale

Lire le dépôt et la stratégie de build avant de lancer. En cas d'échec, localiser précisément la phase : clone, build, push ou consommation de l'image. Une fois le build terminé, confirmer la présence de l'image dans l'ImageStream puis mettre à jour le déploiement.

Checklist de validation

- ☐ BuildConfig créée ;
- ☐ build terminé avec succès ;
- ☐ ImageStream mis à jour ;
- ☐ déploiement pointe sur la bonne image ;
- ☐ pod final sain ;

Variante examen

Branche Git erronée ou nom du Dockerfile non standard.

Erreurs fréquentes et correction rapide

Build casse dès le clone : vérifier URL, branche, secret Git. Build OK mais ancienne version en ligne : contrôler le lien avec l'image réellement utilisée.

Lab 6 - S2I et customisation d'un builder

Temps cible : 30 min

Objectif : Maîtriser le flux S2I et une adaptation simple d'image builder existante.

Point de départ

Un dépôt de code est fourni. Une image builder S2I doit être utilisée puis ajustée.

Points d'attention

- builder inadapté ;
- scripts assemble/run mal copiés ;
- application qui build mais ne démarre pas.

Tâches à exécuter

29. Créer le build S2I.
30. Personnaliser assemble ou run si le sujet le demande.
31. Relancer le build après correction.
32. Déployer et contrôler les logs applicatifs.

Commandes et vérifications réflexes

- oc new-build --strategy=source ...

- oc start-build <bc> --follow
- oc logs build/<build>
- oc get buildconfig,builds,imagestreams

Solution type minimale

Commencer par faire passer le build avec le builder le plus logique. Ensuite distinguer clairement une panne de build d'une panne de runtime. Si l'image est générée mais que l'application ne démarre pas, le diagnostic bascule sur les logs du pod.

Checklist de validation

- ☐ builder cohérent ;
- ☐ build S2I terminé ;
- ☐ scripts personnalisés pris en compte ;
- ☐ image générée ;
- ☐ application démarrée et logs exploitables ;

Variante examen

assemble cassé au départ ou builder volontairement inadapté.

Erreurs fréquentes et correction rapide

Image générée mais application KO : passer des logs build aux logs runtime. Scripts non pris en compte : vérifier leur emplacement et leurs droits.

Lab 7 - Hooks, triggers et relance contrôlée

Temps cible : 20 min

Objectif : Déclencher et vérifier automatiquement des builds ou rollouts.

Point de départ

Une application existe déjà. Il faut ajouter les mécanismes de déclenchement utiles.

Points d'attention

- trigger présent mais jamais activé ;
- hook mal référencé ;
- confusion entre build trigger et rollout trigger.

Tâches à exécuter

33. Ajouter un hook de build ou de déploiement selon l'énoncé.
34. Configurer un trigger image ou config change.
35. Provoquer le changement attendu.
36. Vérifier que le déclenchement réel correspond à l'intention.

Commandes et vérifications réflexes

- oc describe bc/<nom>
- oc describe deploy/<nom>
- oc rollout history deploy/<nom>
- oc get events --sort-by=.metadata.creationTimestamp | tail

Solution type minimale

Toujours expliciter quel événement doit déclencher quoi : un changement de configuration, une nouvelle image, ou une phase de déploiement. Le point clé n'est pas la présence déclarative du trigger, mais la preuve qu'il s'est exécuté.

Checklist de validation

- ☐ hook ou trigger correctement déclaré ;
- ☐ événement déclencheur provoqué ;
- ☐ nouveau rollout ou nouveau build visible ;
- ☐ historique de déploiement compréhensible ;
- ☐ events cohérents ;

Variante examen

Build trigger à distinguer d'un image trigger, ou hook post-déploiement à prouver.

Erreurs fréquentes et correction rapide

Rien ne se passe : vérifier le type exact de trigger, l'objet concerné et l'événement réellement produit.

Lab 8 - Templates OpenShift paramétrés

Temps cible : 25 min

Objectif : Produire un template réutilisable puis l'instancier proprement.

Point de départ

Une application connue doit être transformée en template paramétré.

Points d'attention

- paramètre non utilisé ;
- valeur codée en dur ;
- objet associé oublié ;
- template valide mais inutilisable.

Tâches à exécuter

37. Créer le template avec paramètres nom, image, port, mémoire et hostname.
38. Inclure les objets indispensables.
39. Traiter puis appliquer une instance.
40. Vérifier que les paramètres sont bien passés.

Commandes et vérifications réflexes

- oc process -f template.yaml -p APP_NAME=demo
- oc apply -f -
- oc get all
- oc get route -o yaml

Solution type minimale

Construire un template compact mais complet. Le sujet récompense moins la sophistication que la bonne consommation des paramètres. Utiliser oc process pour inspecter le rendu avant application reste le réflexe clé.

Checklist de validation

- ☐ template valide ;
- ☐ tous les paramètres sont déclarés ;
- ☐ les objets attendus sont générés ;
- ☐ aucune valeur sensible n'est restée codée en dur ;
- ☐ le rendu final correspond à l'instance demandée ;

Variante examen

Ajout d'une route optionnelle ou d'une limite mémoire paramétrable.

Erreurs fréquentes et correction rapide

Template processé mais incohérent : relire les placeholders et vérifier que chaque paramètre est réellement consommé.

Lab 9 - Helm puis Kustomize

Temps cible : 30 min

Objectif : Être à l'aise sur un flux chart puis overlay.

Point de départ

Un chart existe déjà mais nécessite adaptation par values et patchs ciblés.

Points d'attention

- mauvais namespace cible ;
- patch Kustomize non appliqué ;
- conflit entre Helm values et overlay.

Tâches à exécuter

41. Installer ou rendre le chart avec le bon fichier values.
42. Modifier l'exposition, l'image ou les ressources via values.
43. Appliquer ensuite une surcouche Kustomize.
44. Vérifier l'état final obtenu.

Commandes et vérifications réflexes

- helm template ...
- helm install/upgrade ...
- oc kustomize ...
- oc get all

Solution type minimale

Helm d'abord pour le rendu de base, Kustomize ensuite pour les ajustements ciblés. Garder le patch léger et vérifier qu'il cible le bon nom, le bon kind et le bon namespace.

Checklist de validation

- ☐ chart rendu ou installé avec les bonnes values ;
- ☐ namespace correct ;
- ☐ patch appliqué sur le bon objet ;
- ☐ état final conforme ;
- ☐ pas de conflit entre la base et l'overlay ;

Variante examen

Patch sur une probe, une image ou des ressources CPU/mémoire.

Erreurs fréquentes et correction rapide

Rien ne change : vérifier la cible du patch et la ressource réellement générée par Helm.

Lab 10 - Pipeline Tekton simple à exécuter et dépanner

Temps cible : 30 min

Objectif : Lire, lancer, suivre et corriger un workflow CI/CD simple.

Point de départ

Les Task et la Pipeline existent déjà partiellement.

Points d'attention

- workspace absent ;
- image de task erronée ;
- secret Git ou registry non monté ;
- relance sans correction réelle.

Tâches à exécuter

45. Créer les ressources manquantes : PipelineRun, Workspace, Secret ou ServiceAccount si nécessaire.
46. Démarrer le pipeline.
47. Suivre les logs et identifier l'étape en échec.
48. Corriger l'entrée ou la référence fautive puis relancer.

Commandes et vérifications réflexes

- oc get pipelineruns,taskruns
- tkn pipelinerun logs -f <nom>
- oc describe pipelinerun <nom>
- oc get events

Solution type minimale

Suivre une méthode fixe : lire, lancer, localiser l'échec, corriger le strict nécessaire puis relancer. Si l'échec intervient avant la première task, vérifier en priorité ServiceAccount, permissions et workspace.

Checklist de validation

- ☐ PipelineRun présente ;
- ☐ TaskRuns créées ;
- ☐ logs lisibles ;
- ☐ cause d'échec localisée ;
- ☐ pipeline corrigée et terminée ;

Variante examen

Workspace PVC manquant ou secret registry absent.

Erreurs fréquentes et correction rapide

Pipeline bloquée très tôt : regarder describe + events avant de modifier les tasks.

Lab 11 - Application déployée via Operator

Temps cible : 15 min

Objectif : Savoir consommer un Operator et créer la ressource attendue.

Point de départ

L'Operator est disponible ou déjà installé selon le lab.

Points d'attention

- mauvaise CR apiVersion ;
- namespace d'installation confus ;
- attente trop courte avant validation.

Tâches à exécuter

49. Vérifier l'Operator pertinent.
50. Appliquer une Custom Resource simple.
51. Contrôler les ressources créées et la santé de l'application.

Commandes et vérifications réflexes

- oc get csv,subscriptions
- oc get crd | grep -i ...
- oc get <cr-kind>
- oc describe <cr-kind>/<nom>

Solution type minimale

Lire la CR officielle avant d'écrire. Les erreurs les plus coûteuses portent sur apiVersion, kind et namespace. Une fois la CR acceptée, attendre juste ce qu'il faut pour observer les ressources créées.

Checklist de validation

- ☐ Operator visible ;
- ☐ CRD présente ;
- ☐ CR appliquée sans erreur ;
- ☐ ressources descendantes créées ;
- ☐ état final compréhensible ;

Variante examen

Operator déjà installé dans un autre namespace ou CR avec nom attendu imposé.

Erreurs fréquentes et correction rapide

CR ignorée : vérifier immédiatement apiVersion, kind et namespace de l'Operator.

Lab 12 - Dépannage transversal

Temps cible : 40 min

Objectif : Traiter un lot de pannes mêlant build, image, service et configuration.

Point de départ

Plusieurs objets existent déjà mais le rendu final n'est pas conforme.

Points d'attention

- corriger trop tôt les symptômes visibles ;
- négliger les events ;
- oublier le registre interne.

Tâches à exécuter

52. Identifier les symptômes prioritaires.
53. Réparer d'abord ce qui bloque la création d'image ou le démarrage des pods.
54. Réparer ensuite l'exposition réseau.
55. Faire une validation complète de bout en bout.

Commandes et vérifications réflexes

- oc get all,cm,secret,is,builds
- oc describe ...
- oc logs ...
- oc get events --sort-by=.lastTimestamp

Solution type minimale

Adopter un ordre fixe : build et image d'abord, démarrage des pods ensuite, exposition réseau après, validation finale à la fin. L'objectif n'est pas de tout relire : il faut remonter à la première cause bloquante et traiter le reste dans le bon ordre.

Checklist de validation

- ☐ cause racine identifiée ;
- ☐ image ou build débloquée ;
- ☐ pods sains ;
- ☐ service et route corrects ;
- ☐ validation finale complète réalisée ;

Variante examen

Deux ou trois pannes combinées, dont une fausse piste visible mais secondaire.

Erreurs fréquentes et correction rapide

Le temps s'évapore quand on répare la route alors que le pod ne démarre pas encore. Revenir à la première cause bloquante.

4. Dépannage rapide et séquence de diagnostic

Cette section prolonge la table de pannes de la V4 en lui donnant une lecture plus opérationnelle : premier réflexe, cause probable et correction minimale.

Symptôme	Cause probable	Réflexe de diagnostic	Correction minimale
Pod en CrashLoopBackOff	commande d'entrée, variable absente ou port attendu différent	oc describe pod ; oc logs ; vérifier env et mount	corriger le démarrage avant toute autre action
Pod Running mais non Ready	readiness probe incorrecte	oc describe pod ; comparer path et port avec l'application	adoucir la probe ou corriger le port
Route répond en erreur	Service ou targetPort	oc get svc,route,endpoints	aligner targetPort et

	incohérent		endpoints
Build échoue dès le clone Git	URL, branche ou secret Git erroné	oc logs build/<nom>	corriger source ou credentials
Build OK mais ancienne version déployée	déploiement toujours lié à une ancienne image	contrôler ImageStreamTag et image du Deployment	pointer vers la bonne image puis relancer si nécessaire
Image importée mais pas de rollout	trigger absent ou non activé	oc describe deploy	ajouter ou comprendre le trigger adapté
Secret créé mais application aveugle	mauvais nom de clé ou mountPath	oc describe deploy ; oc exec dans le pod	corriger le nom de clé ou le chemin
S2I produit une image qui ne démarre pas	scripts assemble ou run, ou builder inadapté	lire logs build puis logs runtime	séparer panne de build et panne d'exécution
Pipeline Tekton échoue avant la première task	ServiceAccount, Workspace ou permission manquante	oc describe pipelinerun ; tkn logs	ajouter l'objet manquant ou la permission
Template processé mais invalide	paramètres non déclarés ou objets manquants	oc process -f template.yaml	corriger placeholders et objets
Operator installé mais CR ignorée	apiVersion/kind erroné ou namespace incorrect	oc get crd ; vérifier la CR officielle	réécrire la CR avec les bons identifiants
Registry interne inaccessible	authentification ou configuration registry	vérifier login, secret et référence registry	corriger le secret ou la référence d'image
Sidecar qui bloque tout	healthcheck ou commande du sidecar défaillante	isoler les logs par conteneur	corriger d'abord le sidecar
Kustomize n'applique pas le patch	cible mal identifiée	vérifier name, namespace et kind du patch	aligner exactement la cible
Hook configuré mais jamais exécuté	mauvais type de hook ou mauvais moment de déclenchement	oc describe build ou deploy	provoquer l'événement attendu

Séquence universelle de diagnostic

- 1) oc get all pour situer rapidement le problème ;
- 2) oc describe de la ressource fautive pour lire l'état et les events associés ;
- 3) logs du pod, du build ou du pipeline ;
- 4) events triés par date ;
- 5) correction minimale ;
- 6) revalidation immédiate.

Ordre gagnant - En dépannage transversal, l'ordre des actions vaut souvent plus que la profondeur du diagnostic. On traite d'abord ce qui empêche l'image d'exister, ensuite ce qui empêche le pod de démarrer, puis ce qui empêche l'application d'être exposée.

5. Examens blancs A et B

Examen blanc A - 3 heures

Pondération indicative d'entraînement. Le score officiel réel relève de Red Hat ; la table ci-dessous sert uniquement à structurer la pratique.

Tâche	Poids	Validation attendue
Créer un projet, déployer une application simple, configurer Service, Route et probes.	18	résultat observable conforme et contrôle par oc
Créer ConfigMap et Secret, puis les injecter proprement.	14	résultat observable conforme et contrôle par oc
Créer un ImageStream personnalisé et déployer depuis ce flux.	14	résultat observable conforme et contrôle par oc
Construire une image depuis un dépôt Git avec BuildConfig.	16	résultat observable conforme et contrôle par oc
Configurer un build S2I et corriger un échec de build fourni.	18	résultat observable conforme et contrôle par oc
Mettre en place un trigger d'image puis démontrer le rollout.	10	résultat observable conforme et contrôle par oc
Créer un template paramétré pour l'application.	14	résultat observable conforme et contrôle par oc
Déployer la même application via Helm puis ajuster une valeur demandée.	12	résultat observable conforme et contrôle par oc
Appliquer un patch Kustomize sur le déploiement.	10	résultat observable conforme et contrôle par oc
Exécuter un PipelineRun Tekton fourni et corriger l'étape qui échoue.	16	résultat observable conforme et contrôle par oc
Traiter deux pannes transversales fournies par l'énoncé.	18	résultat observable conforme et contrôle par oc

Règle d'exécution

Lire tout le sujet pendant 10 à 15 minutes, marquer les tâches rapides, sécuriser les points les plus accessibles, puis revenir aux scénarios de correction plus risqués.

Feuille de score recommandée

- ☐ temps passé par tâche ;
- ☐ résultat : réussi / partiel / échoué ;
- ☐ cause d'échec ;
- ☐ commande décisive ;
- ☐ action à rejouer au prochain entraînement ;

Examen blanc B - 3 heures

56. Sujet 1. Application multi-conteneurs avec volume partagé et exposition réseau correcte.
57. Sujet 2. Image custom bâtie à partir d'un Containerfile et publiée dans le registre interne.
58. Sujet 3. Customisation d'un builder S2I existant puis rebuild.
59. Sujet 4. Hook ou trigger à ajouter puis à vérifier.
60. Sujet 5. Template OpenShift multi-objets.
61. Sujet 6. Pipeline Tekton partiellement fournie à terminer.
62. Sujet 7. Déploiement d'une application gérée par Operator.
63. Sujet 8. Lot final de dépannage.

Objectif du blanc B : varier les combinaisons sans changer les gestes fondamentaux.

Boucle de progression - Après chaque blanc, note au moins trois erreurs récurrentes, puis rejoue seulement les blocs concernés dans les 48 heures. La progression vient moins d'un nouveau sujet que d'une meilleure exécution du même sujet.

6. Plans d'entraînement

Plan intensif sur 7 jours

Jour	Bloc	Labs	Cible
J1	Socle	Labs 1 + 3	déployer vite et sans erreur de namespace
J2	Images	Labs 4 + 5	maîtriser ImageStreams, builds et registry
J3	S2I / triggers	Labs 6 + 7	corriger un build puis prouver le déclenchement
J4	Packaging	Labs 8 + 9	template, Helm, Kustomize sans hésitation
J5	Pipelines / Operator	Labs 10 + 11	lire vite et corriger le strict nécessaire
J6	Dépannage	Lab 12 + section pannes	réduire le temps de diagnostic
J7	Simulation	Blanc A ou B	tenir 3 heures avec méthode

Plan express sur 3 jours

- Jour 1 : Labs 1, 3, 4 et 5 ;
- Jour 2 : Labs 6, 7, 9 et 10 ;
- Jour 3 : Lab 12 puis un examen blanc complet.

Plan progressif sur 14 jours

Version recommandée pour les profils qui veulent alterner apprentissage, répétition et simulation : 8 jours de labs unitaires, 3 jours de rejouabilité ciblée, 2 jours de blancs, 1 jour de consolidation finale.

7. Commandes réflexes

Les 10 commandes vitales

- oc project <ns>
- oc get all -n <ns>
- oc get route,svc,endpoints
- oc describe pod/<pod>
- oc describe deploy/<deploy>
- oc logs <pod>
- oc logs build/<build>
- oc get events --sort-by=.metadata.creationTimestamp | tail -n 30
- oc rollout status deploy/<name>
- tkn pipelinerun logs -f <name>

Commandes classées par domaine

Pods / Deployments

- oc get pods -o wide
- oc describe pod/<pod>
- oc rollout history deploy/<name>

Réseau

- oc get svc,route,endpoints
- oc get route -o yaml

Configuration

- oc get cm,secret
- oc exec <pod> -- env | sort
- oc exec <pod> -- cat /etc/config/app.properties

Builds / Images

- oc start-build <bc> --follow
- oc get builds
- oc get is,istag

Templates / Packaging

- oc process -f template.yaml -p KEY=VALUE
- helm template ...
- oc kustomize ...

Tekton

- oc get pipelineruns,taskruns
- oc describe pipelinerun <nom>
- tkn pipelinerun logs -f <nom>

Operators

- oc get csv,subscriptions
- oc get crd | grep -i ...
- oc describe <cr-kind>/<nom>

Conclusion opérationnelle

La réussite sur EX288 repose sur une combinaison simple mais exigeante : comprendre vite, corriger peu, prouver le résultat. La meilleure préparation n'est pas de lire davantage, mais d'exécuter les mêmes gestes jusqu'à ce qu'ils deviennent fluides, reproductibles et validés.

Dernier contrôle - Révision finale avant inscription : relire la page EX288 officielle, confirmer la version visée, vérifier les objectifs publiés et exécuter au moins un blanc dans des conditions proches de l'épreuve.